FYEO

Theoriq

Repo: https://github.com/chain-ml/tq-oracle

Security Review Update: March 8, 2026

Reviewer: balthasar@gofyeo.com

Security Code Review TQ Oracle

New security issues, 8

After the development team implemented the latest updates, FYEO conducted a review of the modifications. The primary goal of this evaluation was to ensure the continued robustness of the program's security features, safeguarding user funds and maintaining the overall integrity of the program.

General Updates:

This update significantly expands the TQ Oracle system's capabilities to support multi-chain DeFi position tracking and cross-chain bridge reconciliation. The changes introduce support for tracking assets across HyperLiquid's ecosystem (HyperEVM and HyperCore chains) alongside the existing Ethereum mainnet support, while also adding several new protocol integrations for more comprehensive vault position monitoring.

The update adds three major new asset adapters for discovering positions: a HyperCore adapter that fetches portfolio net asset values directly from HyperLiquid's API for both native vaults and sub-accounts, a Morpho Blue adapter that tracks lending positions including supply, borrow, and collateral with real-time interest accrual calculations, and several withdrawal queue adapters for handling pending unstaking positions in protocols like Lido's wstETH withdrawals and Ethena's sUSDe cooldown mechanism. Additionally, the ERC4626 vault adapter now supports applying market discounts to account for illiquidity or withdrawal fees when converting vault shares to underlying assets. The Uniswap V4 adapter was enhanced to calculate and include uncollected trading fees alongside the liquidity positions. A completely new bridge adapter system was introduced to track in-flight cross-chain transfers, with implementations for Circle's CCTP protocol (tracking USDC transfers between Ethereum, Arbitrum, Base, and HyperEVM) and the native HyperEVM-HyperCore bridge, ensuring accurate TVL calculations even when funds are temporarily in transit between chains. The Chainlink price adapter was generalized to support non-ETH base assets by allowing configuration of alternative USD price feeds for two-hop pricing conversions. Throughout the codebase, new constants, configuration options, and ABIs were added to support these integrations, along with enhanced multi-chain asset collection pipelines that can aggregate positions across different blockchain networks into a unified report.

Block Fetch Missing RPC Wrapper and Caching

Finding ID: FYEO-TQO-01
Severity: **Low**
Status: **Remediated**

Description

The block fetch in `_get_cooldown_state()` has two problems:

1. **Missing RPC wrapper**: Uses synchronous `self.w3.eth.get_block()` instead of `await self._rpc(...)`, bypassing rate limiting, timeout protection, retry logic, and RPC delay/jitter. This blocks the event loop and doesn't respect the adapter's RPC throttling configuration.
2. **Missing caching**: Fetches block timestamp on every call without caching. When processing multiple subvaults at the same block, this causes redundant RPC calls.

Proof of Issue

File name: src/tq_oracle/adapters/asset_adapters/rwa/susde_cooldown.py
Line number: 168

```
block_info = self.w3.eth.get_block(self.block_number)
current_timestamp = block_info["timestamp"]
```

Severity and Impact Summary

The missing `_rpc()` wrapper could cause RPC rate limit issues, hang indefinitely if the node is slow, and degrades performance when processing multiple subvaults concurrently. The missing cache multiplies these issues when checking multiple subvaults.

Recommendation

1. Add `_block_timestamp_cache`
2. Implement `_get_block_timestamp()`
3. Replace the direct call with `await self._get_block_timestamp()`

This ensures proper async handling, RPC throttling, and eliminates redundant calls.

Error Handling Violates Fail-Fast Architecture

Finding ID: FYEO-TQO-02
Severity: **Low**
Status: **Remediated**

Description

Multiple adapters wrap individual fetch operations in try/except blocks that log errors but continue processing. This violates the documented architecture principle: “Error handling: Fail-fast - any adapter failure raises, no partial results.”. This pattern appears in the HyperCore adapter and the Morpho Blue adapter.

If some operations fail while others succeed, the adapter returns partial data, leading to incorrect TVL calculations. In a consensus system, if different operators experience different partial failures (due to network issues or RPC timeouts), they will report different TVL values and fail to reach consensus.

Proof of Issue

File name: src/tq_oracle/adapters/asset_adapters/hypercore.py
Line numbers: 366

```
# Fetch native vault NAVs
for vault_config in self._chain_config.vaults:
    try:
        ...
    except Exception as e:
        logger.error("Failed to fetch HyperCore vault %s: %s", ...
        # Continues to next vault instead of raising

# Similar pattern repeated for subaccounts and subvault addresses
```

File name: src/tq_oracle/adapters/asset_adapters/morpho_blue.py
Line numbers: 562

```
for market_name, market_config in self.markets.items():
    try:
        ...
    except Exception as e:
        logger.error("Morpho Blue market %s failed for %s: %s", ...
        # Continues to next market instead of raising
```

Severity and Impact Summary

Partial failures lead to incorrect TVL calculations that may differ across operators, breaking consensus. Specific risks:

- If any configured vaults/markets fail, the adapter silently returns data for only some, severely underreporting TVL
- Different operators experiencing different RPC failures will report different TVL values
- Silent underreporting makes debugging difficult since errors are only logged, not raised
- Violates the explicit architecture constraint that adapters should fail-fast

Recommendation

Remove try/except blocks and let exceptions propagate per the architecture guide. If partial failure tolerance is required, it should be implemented at the pipeline level with explicit configuration, not silently within individual adapters.

Chainlink Staleness Check Edge Case

Finding ID: FYEO-TQO-03
Severity: **Informational**
Status: **Remediated**

Description

In `_get_feed_price()`, staleness is checked with `price_age > self.staleness_threshold` where `price_age = block_timestamp - updated_at`. If `block_timestamp < updated_at` due to clock skew or chain reorganization, `price_age` becomes negative. The check `price_age > threshold` will pass (negative < positive), incorrectly accepting future-dated prices as fresh.

Proof of Issue

File name: src/tq_oracle/adapters/price_adapters/chainlink.py
Line number: 201

```
# Validation 4: Check staleness threshold
price_age = block_timestamp - updated_at
if price_age > self.staleness_threshold:
    raise ValueError(
        f"Chainlink {label} price stale: age={price_age}s exceeds "
        f"threshold={self.staleness_threshold}s (updatedAt={updated_at}, "
        f"blockTimestamp={block_timestamp})"
    )
```

Severity and Impact Summary

While clock skew or reorgs are rare, accepting future-dated prices could indicate data integrity issues. The negative age would be logged in debug output, but the validation would not catch the anomaly.

Recommendation

Add validation to ensure `price_age >= 0` before checking staleness, or use absolute value with appropriate error handling for negative ages.

ERC4626 Market Discount Validation Missing

Finding ID: FYEO-TQO-04
Severity: **Informational**
Status: **Remediated**

Description

The `discount_tenths_bps` parameter is cast to int without validation. Negative values or values >= 100000 would cause incorrect discount calculations. The discount is applied as `(100000 - discount_tenths_bps) / 100000`, so values outside the valid range could result in negative amounts or excessive discounts.

Proof of Issue

File name: src/tq_oracle/adapters/asset_adapters/rwa/erc4626_vault.py
Line number: 226, 353

```
discount_tenths_bps = int(vault_config.get("market_discount", 0))

# Later used in calculation:
if discount_tenths_bps > 0:
    discounted_assets = int(assets) * (100000 - discount_tenths_bps) // 100000
```

Severity and Impact Summary

Invalid discount values could cause incorrect asset valuations. Negative values would inflate amounts, while values >= 100000 would result in zero or negative amounts, both leading to incorrect TVL calculations.

Recommendation

Add validation to ensure `discount_tenths_bps` is within the valid range before applying the discount calculation.

Inefficient HTTP Session Management

Finding ID: FYEO-TQO-05
Severity: **Informational**
Status: **Remediated**

Description

The adapter creates a new `aiohttp.ClientSession` for every API call in `_fetch_portfolio_nav` and `_fetch_vault_nav`. This prevents connection pooling and creates unnecessary TCP connection overhead. For `fetch_all_assets` which may query dozens of vaults/subaccounts, this results in dozens of separate TCP handshakes. While not a correctness issue, this impacts performance and could contribute to rate limiting or timeouts when fetching many positions.

Proof of Issue

File name: src/tq_oracle/adapters/asset_adapters/hypercore.py
Line number: 111

```
async def _fetch_portfolio_nav(self, address: str) -> int:
    # ...
    async with aiohttp.ClientSession(timeout=API_TIMEOUT) as session:
        async with session.post(url, json=payload) as response:
            # ...

async def _fetch_vault_nav(self, vault_address: str) -> int:
    # ...
    async with aiohttp.ClientSession(timeout=API_TIMEOUT) as session:
        async with session.post(url, json=payload) as response:
            # ...
```

Severity and Impact Summary

Performance issue, not correctness. In a multi-validator consensus system, slower execution doesn't affect correctness, just increases latency.

Recommendation

Create a shared session in `__init__` and reuse it across all API calls.

Missing Validation for Negative Equity in Vault Parsing

Finding ID: FYEO-TQO-06
Severity: **Informational**
Status: **Remediated**

Description

The `_fetch_vault_nav` method in the HyperCore adapter parses equity values from API responses but does not validate that the equity is non-negative before conversion. This is inconsistent with `_parse_history_point` which explicitly checks `value >= 0`.

Negative amounts are intentionally supported in the system for representing debt positions (MorphoBlue borrows, StakeWise debt). The aggregation logic in `compute_total_aggregated_assets()` sums amounts, allowing negative values to offset positive ones.

The question is whether HyperCore portfolio equity can legitimately be negative (representing net debt/losses) or if this indicates malformed API data.

Proof of Issue

File name: src/tq_oracle/adapters/asset_adapters/hypercore.py
Line number: 295

```
try:
    equity = Decimal(str(equity_str))
except (TypeError, ValueError, InvalidOperation):
    raise ValueError(f"Invalid equity value for vault {vault_address}:
{equity_str}")

# Convert to wei using Decimal for precision
nav_usdc_6_decimals = int(equity * USDC_DECIMAL_SCALE)
nav_wei = nav_usdc_6_decimals * DECIMAL_MULTIPLIER
```

Severity and Impact Summary

The system architecturally supports negative amounts for debt positions. The inconsistency with `_parse_history_point` suggests HyperCore portfolios should not have negative equity, but this depends on business requirements. If negative equity is valid (margin debt exceeds collateral), no fix needed. If invalid, add validation.

Recommendation

Clarify business requirements: - If HyperCore portfolios can have negative equity (net debt): No change needed, remove validation from `_parse_history_point` for consistency - If negative equity is invalid: Add validation `if equity < 0: raise ValueError(...)` at line 294, consistent with `_parse_history_point`

Negative TVL Possible in Multi-Chain Reconciliation

Finding ID: FYEO-TQO-07
Severity: **Informational**
Status: **Remediated**

Description

In `collect_multi_chain_assets()`, the calculation `net_tvl = gross_tvl - inflight_total` can become negative if `inflight_total > gross_tvl`. This could occur if bridge reconciliation double-counts transfers or if timing issues cause inflated in-flight amounts. No validation prevents negative TVL from being reported.

Proof of Issue

File name: src/tq_oracle/pipeline/multi_chain.py
Line number: 289

```
# 3. Calculate net TVL
result.net_tvl = result.gross_tvl - result.inflight_total
```

Severity and Impact Summary

Negative TVL is nonsensical and indicates a logic error in bridge reconciliation. While unlikely under normal operation, edge cases in timing or configuration could trigger this condition, leading to invalid oracle reports.

Recommendation

Add validation to ensure `net_tvl >= 0` and raise an error or warning if the condition is violated. Investigate and log details when in-flight amounts exceed gross TVL.

Subvault Address Iteration Without Deduplication

Finding ID: FYEO-TQO-08
Severity: **Informational**
Status: **Open**

Description

The `fetch_all_assets` method queries three separate configuration lists without deduplication: 1. `vaults` (uses `_fetch_vault_nav` with `vault_address`) 2. `subaccounts` (uses `_fetch_portfolio_nav` with `master_address`) 3. `subvault_addresses` (uses `_fetch_portfolio_nav` directly)

If the same address appears in multiple lists, its NAV will be fetched and counted multiple times, inflating TVL.

Proof of Issue

File name: src/tq_oracle/adapters/asset_adapters/hypercore.py
Line number: 366

```
# Fetch native vault NAVs
for vault_config in self._chain_config.vaults:
    nav_wei = await self._fetch_vault_nav(vault_config.vault_address)
    all_assets.append(AssetData(...))

# Fetch sub-account NAVs
for subaccount_config in self._chain_config.subaccounts:
    nav_wei = await self._fetch_portfolio_nav(subaccount_config.master_address)
    all_assets.append(AssetData(...))

# Also fetch for subvault addresses in chain config
for subvault in self._chain_config.subvault_addresses:
    nav_wei = await self._fetch_portfolio_nav(subvault)
    all_assets.append(AssetData(...))
```

Severity and Impact Summary

Duplicate addresses would cause double/triple counting, but this requires misconfiguration. The risk is real if configuration is not carefully validated. Similar adapters (ERC4626) have duplicate detection, suggesting this is a known concern.

Recommendation

Add validation at initialization or before fetching to detect duplicate addresses across all three lists.

Commit Hash Reference:

For transparency and reference, the security review was conducted on the specific commit hash for the Theoriq repository. The commit hash for the reviewed versions is as follows:

d884e4074445dfee48529d62404e6b0255340daf

Remediations were submitted with the commit hash: c879742900c8ec93eabc81b2d3093c21f88ab7d4

Conclusion:

In conclusion, the security aspects of the Theoriq program remain robust and unaffected by the recent updates. Users can confidently interact with the protocol, assured that their funds are well-protected. The commitment to security exhibited by the development team is commendable, and we appreciate the ongoing efforts to prioritize the safeguarding of user assets.